

C++ Vector Practice Problems & Solutions

A collection of foundational C++ vector problems and in-place implementations.

1. Find the Maximum and Minimum

Goal: Take a vector of integers and find the largest and smallest numbers.

Key Concepts: `std::vector`, range-based for loop, basic comparison.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> nums = {12, 45, 7, 89, 23, 3, 56};

    int maxVal = nums[0];
    int minVal = nums[0];

    for (int num : nums) {
        if (num > maxVal) maxVal = num;
        if (num < minVal) minVal = num;
    }

    cout << "Max: " << maxVal << "\nMin: " << minVal << "\n";
    return 0;
}
```

2. Reverse a Vector in Place

Goal: Reverse the elements of a vector in-place using two pointers without extra space.

Key Concepts: Vector indexing, swap(), two-pointer approach.

```
#include <iostream>
#include <vector>
#include <utility>
using namespace std;

int main() {
    vector<int> numbers = {1, 2, 3, 4, 5, 6};

    int left = 0;
    int right = numbers.size() - 1;

    while (left < right) {
        swap(numbers[left], numbers[right]);
        left++;
        right--;
    }

    for (int num : numbers) {
        cout << num << " ";
    }
    cout << "\n";
    return 0;
}
```

Ideas for Improvement

Write a template method (showVector) to display a generic vector.

3. Remove Even Numbers

Goal: Filter a vector of integers to remove all even numbers.

Key Concepts: `vector::erase()`, iterators, modulo operator `%`, operator `[]`

Version 1 – Using iterator

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> nums = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    for (auto it = nums.begin(); it != nums.end(); ) {
        if (*it % 2 == 0) {
            it = nums.erase(it); // erase returns the next valid iterator
        } else {
            ++it;
        }
    }

    for (int num : nums) { cout << num << " "; }
    cout << "\n";
}
```

Version 2 – Using fast/slow indices

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector nums = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

    int slow = 0; // Points to where the next odd number should go

    for (int fast = 0; fast < nums.size(); ++fast) {
        if (nums[fast] % 2 != 0) { // Keep odd numbers
            nums[slow] = nums[fast];
            slow++;
        }
    }

    // Trim the vector to keep only the odd numbers
    nums.resize(slow);

    for (int num : nums) {
        cout << num << " ";
    }
    cout << "\n";
}
```

4. Remove Duplicates from a Sorted Vector (Fast / Slow Index Approach)

Goal: Given a sorted vector, remove all duplicates in-place without using `std::unique`, using the fast/slow index technique.

Key Concepts: Fast and slow pointer pattern, `vector::resize()`.

```
#include <iostream>
#include <vector>
using namespace std;

void removeDuplicates(vector<int>& nums) {
    if (nums.empty()) return;

    int slow = 0; // Points to the position of the last confirmed unique element

    for (int fast = 1; fast < nums.size(); ++fast) {
        // When fast finds a new unique value:
        if (nums[fast] != nums[slow]) {
            slow++; // Advance slow pointer
            nums[slow] = nums[fast]; // Overwrite with the unique value
        }
    }

    // Resize vector to trim off extra remaining elements
    nums.resize(slow + 1);
}

int main() {
    vector<int> nums = {1, 1, 2, 2, 2, 3, 4, 4, 5};

    removeDuplicates(nums);

    for (int num : nums) {
        cout << num << " ";
    }
    cout << "\n";
    return 0;
}
```

5. Running Sum / Prefix Sum

Goal: Transform a vector [a, b, c, d] into [a, a+b, a+b+c, a+b+c+d].

Key Concepts: In-place array mutation, accumulation.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> nums = {3, 1, 4, 2, 5};

    for (int i = 1; i < nums.size(); ++i) {
        nums[i] += nums[i - 1];
    }

    for (int num : nums) {
        cout << num << " ";
    }
    cout << "\n";
    return 0;
}
```

6. Count and Replace Target Value

Goal: Count occurrences of a target value in a vector and replace every occurrence with 0.

Key Concepts: Vector traversal, counter, value replacement.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> nums = {4, 2, 7, 2, 8, 2, 5};
    int target = 2;
    int count = 0;

    for (int &num : nums) {
        if (num == target) {
            count++;
            num = 0;
        }
    }

    cout << "Target " << target << " found " << count << " times.\n";
    cout << "Updated vector: ";
    for (int num : nums) {
        cout << num << " ";
    }
    cout << "\n";
    return 0;
}
```

7. Move Zeros to the Right Side

Goal: Rearrange a vector of integers so that all 0s are shifted to the right end, while maintaining the exact relative order of all non-zero elements.

Key Concepts: Vector traversal, counter, swap operation, in-place vs. extra storage, fast/slow pointer technique.

```
#include <iostream>
#include <vector>
using namespace std;

void showVector(vector<int> v) {
    for (auto item : v) cout << item << " ";
    cout << endl;
}

void moveZerosRight(vector<int>& v) {
    //In-place transformation, no extra space needed, one pass
    //Uses slow-fast pointer approach.
    int slow = 0;
    int fast = 0;
    while (fast < v.size()) {
        if (v[fast] != 0) {
            swap(v[fast], v[slow]);
            slow++;
        }
        fast++;
    }
}

void moveZerosRight2(vector<int>& v) {
    //Simple, extra-space needed, one-pass, less efficient
    vector<int> newv;

    for (auto item : v) {
        if (item != 0) newv.push_back(item);
    }
    newv.resize(v.size());
    v = newv;
}

int main() {
    vector<int> v{ 0, 1, 0, 3, 4, 0, 2, };
    moveZerosRight(v);
    showVector(v);

    v = { 0, 1, 0, 3, 4, 0, 2, };
    moveZerosRight2(v);
    showVector(v);

    cout << "\nAll done!\n";
}
```